CODE WALKTHROUGH · SILT AI DATA ANALYST

The Executor agent:
backend/app/agents/executor.py

The file where **Gemini writes Python code** for one analysis step and the **sandbox runs it**. This document explains it function by function, with the real code and line numbers.

FILE SIZE
173 lines, 2 functions + 1 schema

AI USED
Google Gemini (Llama as backup)

CODE RUNS IN
Docker sandbox, no network

RETRIES PER STEP
3 (`MAX_EXECUTOR_RETRIES`)

1. Where this file sits

Router → Triage → Planner → **Executor** (this file · loops once per step) → Critic → Dashboard

The Planner turns the question into an ordered list of steps. The Executor runs once for every attempt of every step. When all steps are done, the Critic independently re-checks the numbers. The Executor is used on the **standard** route (the full AI pipeline); simpler questions take model-free routes.

2. What happens in one call overview of executor_node

1 **Read the state**
Which question, which step of the plan, and how many times this step has already been tried.

2 **Show progress**
Writes "Executing step 2/4 ..." (or "Retry 1/3 ...") to the database so the Analyses page updates live.

3 **Build the prompt**

Question + dataset profile + results of earlier steps + this step's goal + the last error, if any.

4 **Ask Gemini for code**

`llm_client.call_tool()` forces Gemini to answer through the `submit_code` tool: reasoning, a plain-English formula and a Python script. Falls back to Llama if Gemini fails.

5 **Run it in the sandbox**

`get_runner()` picks the Docker sandbox, the CSV is copied in read-only, and the script runs in a fresh container with no network.

6 **Save the evidence**

One `ExecutionLog` row per attempt: code, stdout, stderr, result, charts, data slice. This is what the Audit trail shows.

7 **Decide what next**

Success: move to the next step. Failure: retry with the error message. Third failure: stop the whole run.

3. Imports and the tool schema

Import	From	Used for
`config`	`app/config.py`	Settings, e.g. `MAX_EXECUTOR_RETRIES` (default 3).
`llm_client`	`app/agents/llm_client.py`	Talks to Gemini (and Llama as backup). Used via `call_tool()`.
`prompts`	`app/agents/prompts.py`	`EXECUTOR_SYSTEM` instructions and `profile_block()` which formats the dataset profile.
`AgentState, update_stage`	`app/agents/state.py`	The shared state passed between agents, and the live-progress writer.
`SessionLocal`	`app/database.py`	Opens a database session (SQLAlchemy).
`ExecutionLog`	`app/models.py`	The table where every code attempt is stored.
`get_runner`	`app/sandbox/runner.py`	Returns the configured sandbox (Docker in production).

CODE_TOOL_SCHEMA (lines 12-34)

Not a function, but the most important definition in the file. It is a JSON Schema that tells Gemini the exact shape of its answer. Because the call is forced to use this tool, Gemini cannot reply with free text; it must fill in three fields:

Field	What Gemini puts there	Where it ends up
`reasoning`	A short explanation of the approach	ExecutionLog, Audit trail
`formula_explanation`	The calculation in plain English, e.g. *growth % = (this month - last month) / last month x 100*	Click-to-inspect panel on the dashboard
`code`	A complete Python script that follows the sandbox contract	Run in the sandbox

backend/app/agents/executor.py : lines 12–34

```
12  CODE_TOOL_SCHEMA = {
13      "type": "object",
14      "properties": {
15          "reasoning": {
16              "type": "string",
17              "description": "Brief explanation of the approach this code takes.",
18          },
19          "formula_explanation": {
```

```
            "type": "string",
            "description": (
                "The calculation this step performs, in one or two plain-English sentences a "
                "non-technical reader could follow (e.g. 'Revenue growth % = (this month's "
                "revenue - last month's revenue) / last month's revenue x 100'). Describe the "
                "actual arithmetic/logic the code below carries out, not a restatement of the g
oal."
            ),
        },
        "code": {
            "type": "string",
            "description": "Complete, self-contained Python script following the sandbox contra
ct.",
        },
    },
    "required": ["reasoning", "formula_explanation", "code"],
}
```

The sandbox contract (in `prompts.py`, part of `EXECUTOR_SYSTEM`) tells Gemini the rules the script must follow:

backend/app/agents/prompts.py : lines 63–85 (SANDBOX_CONTRACT)

```
SANDBOX_CONTRACT = """
SANDBOX CODE CONTRACT (follow exactly):
- Your code runs headless in an isolated container with no network access.
- Load the dataset with: df = pd.read_csv("/workspace/data/input.csv")
- Available libraries: pandas as pd, numpy as np, plotly.express / plotly.graph_objects, matplo
tlib (json, sys are stdlib).
  Import everything you use explicitly at the top of your script.
- To return a structured result, print EXACTLY ONE line of the form:
    print("RESULT_JSON:" + json.dumps(result_dict))
  where result_dict contains the concrete numbers/findings for this step (plain JSON-serializab
le types only).
- To emit a chart, build a Plotly figure and save it with:
    fig.write_json("/workspace/output/<short_name>.json")
  Use a unique, descriptive <short_name> for each chart. Do not use matplotlib for charts that
belong on the dashboard.
  Do not set custom colors, themes, or templates on the figure — leave that to Plotly's default
s.
  The dashboard applies its own consistent dark theme/colorway on top of every chart at render
time.
- So a reviewer can trace this result back to the exact data it came from, ALSO print exactly o
ne
  more line:
    print("DATA_SLICE_JSON:" + json.dumps({"columns": [...], "rows": [[...], ...]}))
  containing the real column names and up to 50 rows of the actual dataframe subset your comput
ation
  is based on (after your filtering/grouping, before collapsing it into the final printed resul
t) —
  not a re-description or a fabricated sample.
- Do not read/write any path outside /workspace. Do not attempt network access, subprocess call
s, or file deletion.
- Keep the script self-contained and runnable top to bottom with no user input.
"""
```

4. Functions in executor.py

`_prior_results_summary`

executor.py · helper

```
def _prior_results_summary(step_results: list[dict]) -> str
```

Turns the results of the steps already finished into short text, so Gemini knows what has been computed and does not recompute it.

INPUTS

`step_results`	List of finished steps. Each is a dict with `step_index`, `description`, `result`.

RETURNS

A string, one block per step: `Step 0: ... result: {...}`. If nothing has run yet, `"(none yet)"`.

STEP BY STEP

1. If the list is empty, return `"(none yet)"` (first step of the plan).
2. Otherwise build one line per step with its index, description and result.
3. Join the lines with newlines and return.

backend/app/agents/executor.py : lines 37–45

```
37  def _prior_results_summary(step_results: list[dict]) -> str:
38      if not step_results:
39          return "(none yet)"
40      lines = []
41      for r in step_results:
42          lines.append(
43              f"Step {r['step_index']}: {r['description']}\n  result: {r.get('result')}"
44          )
45      return "\n".join(lines)
```

> Uses `r.get('result')` instead of `r['result']` so a step without a result does not crash the prompt.

`executor_node`

executor.py · the LangGraph node

```
def executor_node(state: AgentState) -> dict
```

The main function. LangGraph calls it once per attempt of each plan step. It asks Gemini to write the Python code for the current step, runs that code in the sandbox, records the attempt, and

tells the graph what to do next.

INPUTS

`state`	The shared `AgentState`: `question_id`, `question_text`, `plan`, `step_index`, `retry_count`, `last_error`, `profile`, `csv_path`, `step_results`.

RETURNS

A dict with **only the fields that changed**. LangGraph merges it into the state. There are three possible shapes (see section 5).

STEP BY STEP

1. **Lines 49-53. Read the state.** Get the question id, the current step and the retry count. `attempt_number` is human-friendly (1, 2, 3) while `retry_count` starts at 0.
2. **Lines 55-58. Report progress.** Build a message like *"Executing step 2/4: Monthly revenue"*, or *"Retry 1/3 on step 2 ..."* on a retry, and save it with `update_stage()`. `update_stage()` is also a checkpoint: if the user cancelled the run, it stops here.
3. **Lines 60-72. Build the prompt.** An f-string with the business question, the dataset profile (`prompts.profile_block`), the earlier results (`_prior_results_summary`) and this step's description and goal.
4. **Lines 74-80. Add the error on a retry.** If the last attempt failed, its error text is appended with *"Fix the root cause and provide corrected code"*. This is how the model learns from its mistake.
5. **Lines 82-91. Call Gemini.** `llm_client.call_tool()` sends the system prompt `EXECUTOR_SYSTEM` (which contains the sandbox rules), the prompt above and `CODE_TOOL_SCHEMA`. The reply is a dict; the code takes out `code`, `reasoning` and `formula_explanation`.
6. **Lines 93-95. Run in the sandbox.** `get_runner()` returns the Docker runner, `seed_input_data()` copies the CSV in as read-only `input.csv`, and `run()` executes the script and returns a `SandboxResult`.
7. **Lines 97-119. Save the attempt.** Open a database session, insert an `ExecutionLog` row with the code, stdout, stderr, success flag, result, charts, reasoning, formula and data slice, commit, and keep its id. `finally: db.close()` always closes the session, even on an error.
8. **Lines 121-142. If it succeeded:** build a `step_result` dict, add it to the list, move `step_index` forward, and reset `retry_count` and `last_error`.
9. **Lines 145-168. If it failed and this was the last allowed attempt:** record the failed step and return `failed=True` with a readable `failure_reason` that includes the last error.
10. **Lines 170-173. If it failed but retries remain:** return only `retry_count + 1` and the error text. The graph sends the same step back to this function.

backend/app/agents/executor.py : lines 48–95 (prompt, Gemini call, sandbox run)

```
48  def executor_node(state: AgentState) -> dict:
49      question_id = state["question_id"]
50      step_index = state["step_index"]
51      retry_count = state.get("retry_count", 0)
52      attempt_number = retry_count + 1
53      step = state["plan"][step_index]
54
55      stage_detail = f"Executing step {step_index + 1}/{len(state['plan'])}: {step['description']}"
56      if retry_count > 0:
```

```
        stage_detail = f"Retry {retry_count}/{config.MAX_EXECUTOR_RETRIES} on step {step_index + 1}: {step['description']}"
    update_stage(question_id, "executing", stage_detail)

    user_content = f"""Business question: {state['question_text']}

Dataset profile:
{prompts.profile_block(state['profile'])}

Prior step results (already computed, available context — do not recompute them, but you may
reference their numbers in your own analysis if directly relevant):
{_prior_results_summary(state.get('step_results', []))}

Current step ({step_index + 1}/{len(state['plan'])}):
Description: {step['description']}
Goal: {step['goal']}
"""

    if state.get("last_error"):
        user_content += f"""
Your previous attempt at this step failed with this error:
{state['last_error']}

Fix the root cause and provide corrected code.
"""

    output = llm_client.call_tool(
        system=prompts.EXECUTOR_SYSTEM,
        user_content=user_content,
        tool_name="submit_code",
        tool_schema=CODE_TOOL_SCHEMA,
        tool_description="Submit the Python script for this analysis step.",
    )
    code = output["code"]
    reasoning = output.get("reasoning", "")
    formula_explanation = output.get("formula_explanation", "")

    runner = get_runner()
    runner.seed_input_data(question_id, state["csv_path"])
    sandbox_result = runner.run(question_id, code, step_index, attempt_number)
```

backend/app/agents/executor.py : lines 97–119 (save the attempt)

```
    db = SessionLocal()
    try:
        log_row = ExecutionLog(
            question_id=question_id,
            step_index=step_index,
            step_description=step["description"],
            attempt_number=attempt_number,
            code=code,
            stdout=sandbox_result.stdout,
            stderr=sandbox_result.stderr,
            success=sandbox_result.success,
            result_json=sandbox_result.result or {},
            chart_paths_json=sandbox_result.chart_paths,
            reasoning=reasoning,
            formula_explanation=formula_explanation,
```

```
            data_slice_json=sandbox_result.data_slice or {},
        )
        db.add(log_row)
        db.commit()
        db.refresh(log_row)
        log_id = log_row.id
    finally:
        db.close()
```

backend/app/agents/executor.py : lines 121–173 (success, retry or fail)

```
    if sandbox_result.success:
        step_result = {
            "step_index": step_index,
            "description": step["description"],
            "code": code,
            "stdout": sandbox_result.stdout,
            "stderr": sandbox_result.stderr,
            "result": sandbox_result.result,
            "chart_paths": sandbox_result.chart_paths,
            "success": True,
            "reasoning": reasoning,
            "attempts": attempt_number,
            "execution_log_id": log_id,
            "formula_explanation": formula_explanation,
        }
        new_results = state.get("step_results", []) + [step_result]
        return {
            "step_results": new_results,
            "step_index": step_index + 1,
            "retry_count": 0,
            "last_error": None,
        }

    # Failure: either retry, or exhaust retries and fail the whole run.
    error_text = sandbox_result.stderr or "(no stderr captured)"
    if retry_count + 1 >= config.MAX_EXECUTOR_RETRIES:
        step_result = {
            "step_index": step_index,
            "description": step["description"],
            "code": code,
            "stdout": sandbox_result.stdout,
            "stderr": sandbox_result.stderr,
            "result": None,
            "chart_paths": [],
            "success": False,
            "reasoning": reasoning,
            "attempts": attempt_number,
            "execution_log_id": log_id,
        }
        new_results = state.get("step_results", []) + [step_result]
        return {
            "step_results": new_results,
            "failed": True,
            "failure_reason": (
                f"Step {step_index + 1} ('{step['description']}') failed after "
                f"{config.MAX_EXECUTOR_RETRIES} attempts. Last error:\n{error_text}"
            ),
        }
```

```
169
170     return {
171         "retry_count": retry_count + 1,
172         "last_error": error_text,
173     }
```

5. What executor_node returns

LangGraph merges the returned dict into the shared state, then `route_after_executor()` decides the next node.

Outcome	Fields returned	What runs next
Success	`step_results` + this step, `step_index` + 1, `retry_count` = 0, `last_error` = None	Next step, or the Critic if this was the last step
Failed, retries left	`retry_count` + 1, `last_error` = the stderr text	Executor again, same step; Gemini now sees the error
Failed, no retries left	`step_results` + failed step, `failed` = True, `failure_reason`	END: the run stops and the user sees the reason

6. Functions it calls in other files

These are not defined in executor.py, but you need them to explain how Gemini and the sandbox actually work.

`llm_client.call_tool`

llm_client.py · Gemini, then Llama

```
def call_tool(system, user_content, tool_name, tool_schema, tool_description, max_tokens=4096)
-> dict
```

The model call used on line 82. It tries Gemini first and, if Gemini cannot answer, fails over to

Llama so the run does not die mid-way.

INPUTS

`system`	The agent's instructions (`EXECUTOR_SYSTEM`).
`user_content`	The prompt built in `executor_node` .
`tool_name / tool_schema`	`submit_code` and `CODE_TOOL_SCHEMA` : the exact shape the answer must have.

RETURNS

The tool's arguments as a dict, e.g. `{"reasoning": ..., "formula_explanation": ..., "code": ...}` . Raises `ProviderUnavailable` if both providers fail.

STEP BY STEP

1. If a `GEMINI_API_KEY` is set, call `call_tool_gemini()` in a loop.
2. On a 429 (rate limit) or a retryable 5xx error, wait and retry, up to `GEMINI_RATE_LIMIT_RETRIES` (2) times.
3. On any other error, including a reply without the tool call (`MalformedOutput`), stop trying Gemini.
4. If failover is enabled and a `LLAMA_API_KEY` is set, make the same call on Llama.
5. If that fails too, raise `ProviderUnavailable` listing why each provider failed.
6. `RunAborted` (user cancelled, budget exceeded) is re-raised at once and never triggers failover.

backend/app/agents/llm_client.py : lines 381–406 (Gemini try + rate-limit retry)

```
381      if config.GEMINI_API_KEY:
382          attempt = 0
383          while True:
384              started = time.monotonic()
385              try:
386                  out = call_tool_gemini(**kwargs)
387                  LAST_PROVIDER = f"gemini:{config.GEMINI_MODEL}"
388                  return out
389              except runtime.RunAborted:
390                  raise  # cancelled / over budget: not a provider failure, never fail over
391              except errors.ClientError as e:
392                  failure = classify_failure(e)
393                  if e.code == 429 and attempt < config.GEMINI_RATE_LIMIT_RETRIES:
394                      wait_s = _retry_delay_seconds(e, attempt)
395                      attempt += 1
396                      log_provider_event("gemini", tool_name, failure, attempt=attempt, max
_attempts=tries, action="retry",
397                                         wait_s=wait_s, error_type=type(e).__name__, model=
config.GEMINI_MODEL,
398                                         elapsed_ms=_elapsed_ms(started))
399                      time.sleep(wait_s)
400                      continue
401                  failures.append(f"gemini({config.GEMINI_MODEL}): {e}")
402                  reasons.append(_failed("gemini", config.GEMINI_MODEL, e))
403                  log_provider_event("gemini", tool_name, failure, attempt=attempt + 1, max
```

```
_attempts=tries,
404                                     action=next_step, error_type=type(e).__name__, model=c
onfig.GEMINI_MODEL,
405                                     elapsed_ms=_elapsed_ms(started))
406                 break
```

`llm_client.call_tool_gemini`

llm_client.py · the actual Gemini request

```
def call_tool_gemini(system, user_content, tool_name, tool_schema, tool_description, max_token
s=4096) -> dict
```

Makes one request to Google Gemini and **forces** it to answer by calling exactly one named function.

INPUTS

`same as call_tool`	Passed straight through.

RETURNS

The function call's arguments as a dict.

STEP BY STEP

1. `runtime.checkpoint(about_to="llm")` : stop if the run was cancelled or is over budget.
2. Wrap the schema in a `FunctionDeclaration` .
3. Call `client.models.generate_content()` with `mode="ANY"` and `allowed_function_names=[tool_name]` , which means Gemini *must* call that tool.
4. Record token usage and time with `runtime.note_llm_call()` (shown in the run trace and cost).
5. Find the function call in the response and return its arguments.

backend/app/agents/llm_client.py : lines 290–311

```
290     client = get_client()
291     function_decl = types.FunctionDeclaration(
292         name=tool_name,
293         description=tool_description,
294         parameters=tool_schema,
295     )
296
297     response = client.models.generate_content(
298         model=config.GEMINI_MODEL,
299         contents=user_content,
300         config=types.GenerateContentConfig(
301             system_instruction=system,
302             max_output_tokens=max_tokens,
303             tools=[types.Tool(function_declarations=[function_decl])],
304             tool_config=types.ToolConfig(
305                 function_calling_config=types.FunctionCallingConfig(
306                     mode="ANY",
307                     allowed_function_names=[tool_name],
308                 )
309             ),
```

```
310          ),
311      )
```

get_runner

sandbox/runner.py

```
def get_runner() -> SandboxRunner
```

Chooses which sandbox to use from the `SANDBOX_BACKEND` setting.

INPUTS

`(none)`	Reads `config.SANDBOX_BACKEND`, default `"docker"`.

RETURNS

A `DockerSandboxRunner`, or a `SubprocessSandboxRunner` for local development only.

STEP BY STEP

1. `"docker"` returns the Docker runner, the real security boundary.
2. `"subprocess"` is allowed only when `SANDBOX_ALLOW_UNSAFE_SUBPROCESS=true`; it is not isolated.
3. Anything else raises `SandboxUnavailable`. It never quietly falls back to a weaker sandbox.

backend/app/sandbox/runner.py : lines 557–567

```
557  def get_runner() -> SandboxRunner:
558      """The configured runner. Raises SandboxUnavailable instead of ever
559      returning a weaker runner than the one asked for."""
560      backend = (config.SANDBOX_BACKEND or "").strip().lower()
561      if backend == "docker":
562          return DockerSandboxRunner()
563      if backend == "subprocess":
564          runner = SubprocessSandboxRunner()
565          runner.ensure_available()
566          return runner
567      raise SandboxUnavailable(f"Unknown SANDBOX_BACKEND '{backend}'")
```

SandboxRunner.seed_input_data

sandbox/runner.py

```
def seed_input_data(self, question_id: int, source_csv_path: str) -> Path
```

Copies the uploaded CSV into this question's private workspace as `data/input.csv`.

INPUTS

`question_id`	Each question gets its own folder `q<id>`.
`source_csv_path`	Where the uploaded file is stored.

RETURNS

The path of the copy.

STEP BY STEP

1. Create `q<id>/data`, `runs` and `artifacts` folders if needed.
2. Copy the CSV only if it is not already there (later steps reuse it).
3. Make it read-only (`chmod 0o444`) so generated code cannot change the data.

backend/app/sandbox/runner.py : lines 181–190

```
181      def seed_input_data(self, question_id: int, source_csv_path: str) -> Path:
182          ws = self.workspace_for(question_id)
183          dest = ws / "data" / "input.csv"
184          if not dest.exists():
185              shutil.copyfile(source_csv_path, dest)
186              try:
187                  os.chmod(dest, 0o444)
188              except OSError:
189                  pass
190          return dest
```

SandboxRunner.run

sandbox/runner.py

```
def run(self, question_id: int, code: str, step_index: int, attempt: int) -> SandboxResult
```

Runs one script safely and packages everything that came out of it. Called on line 95.

INPUTS

`code`	The Python script from Gemini.
`step_index / attempt`	Used to name this execution, e.g. `s1_a2_3f9c...`.

RETURNS

A `SandboxResult`: `success`, `stdout`, `stderr`, `exit_code`, `result`, `chart_paths`, `data_slice`, `timed_out` and more.

STEP BY STEP

1. Check the run is not cancelled, and that the sandbox is available.
2. Reject the script if it is empty, too large (over 200 KB) or contains a null byte.
3. Create a fresh folder for this execution with an `out` folder for charts.
4. Run it with `_execute()` (Docker), then validate the charts with `_collect_artifacts()`.
5. Always delete the execution folder afterwards (`finally`).
6. Success means exit code 0 and no timeout or cancellation. Add a hint if the process was killed (exit 137).
7. Parse `RESULT_JSON:` and `DATA_SLICE_JSON:` from stdout and record the run in the trace.

backend/app/sandbox/runner.py : lines 259–302

```
259      def run(self, question_id: int, code: str, step_index: int, attempt: int) -> SandboxR
esult:
260          runtime.checkpoint(about_to="sandbox")
261          self.ensure_available()
262          started = time.monotonic()
263          if not isinstance(code, str) or "\x00" in code or len(code.encode("utf-8", "repla
ce")) > config.SANDBOX_MAX_CODE_BYTES:
264              return SandboxResult(False, "", "[sandbox] script rejected: empty, too large
or not text",
265                                   None, None, isolation=self.isolation)
266
267          ws = self.workspace_for(question_id)
268          run_id = f"s{step_index}_a{attempt}_{uuid.uuid4().hex[:8]}"
269          exec_dir = ws / "runs" / run_id
270          out_dir = exec_dir / "out"
271          out_dir.mkdir(parents=True)
272          timeout = runtime.remaining_seconds(float(config.SANDBOX_TIMEOUT_SECONDS))
273          try:
274              raw = self._execute(ws, exec_dir, code, timeout)
275              charts, rejected = self._collect_artifacts(out_dir, ws / "artifacts" / run_i
d)
276          finally:
277              shutil.rmtree(exec_dir, ignore_errors=True)
278
279          success = raw.exit_code == 0 and not (raw.timed_out or raw.cancelled)
280          stderr = raw.stderr
281          if raw.exit_code in (137, -9) and not raw.timed_out:
282              stderr += "\n[sandbox] process was killed (memory or another resource limit)"
283          result = SandboxResult(
284              success=success,
285              stdout=raw.stdout,
286              stderr=stderr,
287              exit_code=raw.exit_code,
288              result=self._parse_stdout(raw.stdout) if success else None,
289              chart_paths=charts if success else [],
290              timed_out=raw.timed_out,
291              data_slice=self._parse_stdout(raw.stdout, DATA_SLICE_MARKER) if success else
None,
292              cancelled=raw.cancelled,
293              output_truncated=raw.overflow,
294              rejected_artifacts=rejected,
295              isolation=self.isolation,
296          )
297          runtime.note_sandbox_run(self.name, step_index, attempt, success, raw.timed_out,
```

```
                                    int((time.monotonic() - started) * 1000),
                                    {"rejected_artifacts": len(rejected), "output_truncate
d": raw.overflow})
        if raw.cancelled:
            runtime.checkpoint()  # raises RunCancelled / DeadlineExceeded
        return result
```

DockerSandboxRunner._execute and _command

sandbox/runner.py · the container

```
def _execute(self, ws, exec_dir, code, timeout) -> _RawRun
```

Writes the script to `main.py` and starts a brand-new Docker container just for it.

INPUTS

`ws / exec_dir`	The question's workspace and this execution's folder.
`timeout`	Seconds left, at most `SANDBOX_TIMEOUT_SECONDS` (60).

RETURNS

A `_RawRun` with stdout, stderr, exit code and flags for timeout, cancellation and overflow.

STEP BY STEP

1. Write `main.py` and allow the container user to write to `out`.
2. Start `docker run` with the locked-down options below.
3. Hand the process to `_supervise()`, with a `kill()` that runs `docker kill` on the container.
4. Always `docker rm -f` the container at the end.

backend/app/sandbox/runner.py : lines 341–371 (the docker run command)

```
    def _command(self, ws: Path, exec_dir: Path, container: str) -> list[str]:
        docker_bin = shutil.which("docker") or "docker"
        fsize = _parse_size(config.SANDBOX_MAX_OUTPUT_BYTES, 256 * 1024 ** 2)
        cmd = [
            docker_bin, "run", "--rm", "--name", container,
            "--network", "none",              # no egress at all: no internet, no host, n
o metadata endpoint
            "--read-only",                    # root filesystem is immutable
            "--cap-drop", "ALL",
            "--security-opt", "no-new-privileges",
            "--pids-limit", str(config.SANDBOX_PIDS_LIMIT),
            "--memory", config.SANDBOX_MEMORY_LIMIT,
            "--memory-swap", config.SANDBOX_MEMORY_LIMIT,  # equal to --memory: no swap
            "--cpus", config.SANDBOX_CPU_LIMIT,
            "--ulimit", f"fsize={fsize}:{fsize}",
            "--ulimit", "nofile=256:256",
            "--ulimit", "core=0:0",
            "--ipc", "none",
            "--tmpfs", f"/tmp:rw,noexec,nosuid,nodev,size={config.SANDBOX_TMPFS_SIZE}",
            "-e", "HOME=/tmp", "-e", "MPLCONFIGDIR=/tmp", "-e", "MPLBACKEND=Agg",
            "-e", "OPENBLAS_NUM_THREADS=1", "-e", "OMP_NUM_THREADS=1",
```

```
            "-e", "PYTHONDONTWRITEBYTECODE=1",
            "-v", f"{ws / 'data'}:/workspace/data:ro",
            "-v", f"{exec_dir / 'out'}:/workspace/output:rw",
            "-v", f"{exec_dir / 'main.py'}:/workspace/main.py:ro",
            "-w", "/tmp",
        ]
        if config.SANDBOX_DOCKER_RUNTIME:
            cmd += ["--runtime", config.SANDBOX_DOCKER_RUNTIME]
        # -I: isolated mode (no user site, no PYTHON* variables, cwd not on sys.path)
        cmd += [config.SANDBOX_IMAGE, "-I", "-B", "/workspace/main.py"]
        return cmd
```

> Key options: `--network none` (no internet), `--read-only`, `--cap-drop ALL`, `no-new-privileges`, memory 512 MB with no swap, 1 CPU, a process limit, the data mounted read-only and only `/workspace/output` writable.

_supervise

sandbox/runner.py · the watchdog

```
def _supervise(proc, kill, timeout: float, output_limit: int) -> _RawRun
```

Watches the running container and kills it if it takes too long, is cancelled, or prints too much.

INPUTS

`proc`	The running `docker run` process.
`kill`	Function that stops the whole container.
`timeout / output_limit`	Time and stdout limits.

RETURNS

A `_RawRun`.

STEP BY STEP

1. Start two `_Pump` threads that read stdout and stderr, keeping at most the limit.
2. Every 0.2 s check: deadline passed? user cancelled? output far over the limit?
3. If any is true, call `kill()`.
4. Add a clear note to stderr, e.g. `[sandbox] execution timed out`, which Gemini then sees on the retry.

backend/app/sandbox/runner.py : lines 122–140

```
def _supervise(proc: subprocess.Popen, kill, timeout: float, output_limit: int) -> _RawRun:
    """Wait for the child while enforcing the wall clock, cancellation and the
    output cap. `kill` must terminate the whole execution, not just `proc`."""
    out, err = _Pump(proc.stdout, output_limit), _Pump(proc.stderr, output_limit)
    out.start()
    err.start()
    deadline = time.monotonic() + timeout
```

```
    timed_out = cancelled = overflow = False
    while proc.poll() is None:
        if time.monotonic() >= deadline:
            timed_out = True
        elif runtime.is_cancel_requested():
            cancelled = True
        elif out.total > output_limit * 4 or err.total > output_limit * 4:
            overflow = True
        if timed_out or cancelled or overflow:
            kill()
            break
        time.sleep(_POLL_SECONDS)
```

`route_after_executor`

graph.py · what runs next

```
def route_after_executor(state: AgentState) -> str
```

Not in executor.py, but it reads what `executor_node` returned and picks the next node.

INPUTS

`state`	The state after the Executor's update was merged.

RETURNS

`"failed"` (end), `"critic"` (all steps done) or `"executor"` (next step or retry).

STEP BY STEP

1. If `failed` is set, end the run.
2. If `step_index` has reached the end of the plan, go to the Critic.
3. Otherwise run the Executor again.

backend/app/agents/graph.py : lines 94–99

```
def route_after_executor(state: AgentState) -> str:
    if state.get("failed"):
        return "failed"
    if state["step_index"] >= len(state["plan"]):
        return "critic"
    return "executor"
```

7. Worked example illustrative values

Question: *"Why did revenue drop in March?"*. The Planner made 4 steps; this is step 2. The numbers below are made up to show the shape of the data at each stage.

A. PROMPT SENT TO GEMINI

B. GEMINI'S TOOL CALL (SUBMIT_CODE)

```
Business question: Why did revenue drop in March?

Dataset profile:
{ "row_count": 1200, "col_count": 6, "columns": [ {"name": "order_date", ...}, {"name": "revenue", ...} ] }

Prior step results (already computed ...):
Step 0: Load data and parse dates
  result: {'rows': 1200, 'months': 6}

Current step (2/4):
Description: Monthly revenue totals
Goal: Compute total revenue per month to locate the drop
```

```
{
  "reasoning": "Group by calendar month and sum revenue.",
  "formula_explanation": "Monthly revenue = sum of revenue for all orders in that month.",
  "code": "import pandas as pd, json\n
           df = pd.read_csv('/workspace/data/input.csv')\n
           df['order_date'] = pd.to_datetime(df['order_date'])\n
           m = df.groupby(df['order_date'].dt.to_period('M'))['revenue'].sum()\n
           print('RESULT_JSON:' + json.dumps({str(k): float(v) for k, v in m.items()}))\n
           print('DATA_SLICE_JSON:' + json.dumps({...}))"
}
```

C. SANDBOX STDOUT

```
RESULT_JSON:{"2026-01": 182300.0, "2026-02": 179950.0, "2026-03": 141200.0}
DATA_SLICE_JSON:{"columns": ["order_date", "revenue"], "rows": [["2026-03-01", 120.0], ...]}
```

D. EXECUTOR_NODE RETURNS

```
{ "step_results": [ ...step 0..., {"step_index": 1, "result": {"2026-03": 141200.0, ...},
                                   "success": True, "attempts": 1, "execution_log_id": 512, ...} ],
  "step_index": 2, "retry_count": 0, "last_error": None }
```

If the script had crashed, for example with `KeyError: 'Revenue'` because of a wrong column name, step d would instead return `{"retry_count": 1, "last_error": "KeyError: 'Revenue' ..."}`, and the next prompt would end with that error and *"Fix the root cause and provide corrected code."*

8. Likely viva questions

Why force Gemini to use a tool instead of normal text?

A tool call returns structured JSON with fixed fields (`reasoning` , `formula_explanation` , `code`). Free text would need fragile parsing, and the model might wrap code in prose or markdown.

Why not just run the generated code with `exec()` ?

The code is written by a model that read a user's question and a user's CSV, so it is treated as hostile input. It runs in a separate Docker container with no network, a read-only filesystem, no Linux capabilities, and CPU, memory, process and time limits.

What happens if the code crashes?

The stderr (error text) is saved and put into the next prompt with "Fix the root cause". Gemini gets up to 3 attempts per step (`MAX_EXECUTOR_RETRIES`). This is the self-correction loop.

What if Gemini is down or rate-limited?

`call_tool()` retries a 429 or 5xx error up to 2 times with a wait, then automatically switches to a Llama model. Only if both fail does the run stop with `ProviderUnavailable` .

How does the result get back from the container?

Through stdout. The script must print one line starting `RESULT_JSON:` and one starting `DATA_SLICE_JSON:` . Charts are saved as Plotly JSON in `/workspace/output` . The runner parses these and validates every file.

Why save an ExecutionLog for failed attempts too?

Auditability. The Audit trail shows every attempt, including the ones that failed and why, so nothing is hidden.

Why does the Executor not compute everything in one script?

The Planner splits the question into small steps. Each step is easier to get right, easier to verify, and a failure only retries one step instead of the whole analysis.

What stops a script from running forever or printing gigabytes?

`_supervise()` kills the container on timeout (60 s default), on cancellation, or when output exceeds the limit.

Generated from the repository source on 16 October 2026. Every code block is copied from the file and line range shown in its caption.